

1. Patrón Repository

El patrón Repository centraliza el acceso a una colección de objetos en memoria (o base de datos). En lugar de esparcir lógica de búsqueda/guardado por todo el código, todo pasa por una sola clase.

En este proyecto, `Repository.java` es genérico:

```
public class Repository<T extends Identificable> {
    private ArrayList<T> lista = new ArrayList<>();

    public void agregar(T objeto) { ... }
    public T buscarPorCodigo(int codigo) { ... } // usa getCodigo() de Identificable
    public void eliminar(T objeto) { ... }
}
```

Con una sola clase podés tener:

- `Repository<Categoria>` — gestiona categorías
- `Repository<Articulo>` — gestiona artículos (y sus subtipos)

Esto anticipa lo que luego es `JpaRepository<T, ID>` en Spring Boot.

2. Cómo se implementa `Identificable`

La interfaz define un único contrato: cualquier objeto que la implemente debe poder devolver su código.

```
// Identificable.java
public interface Identificable {
    int getCodigo();
}
```

Tanto `Categoria.java` como `Articulo.java` la implementan directamente:

```
public class Categoria implements Identificable {
    public int getCodigo() { return codigo; }
}

public class Articulo implements Identificable {
    public int getCodigo() { return codigo; }
}
```

Esto le permite al repositorio usar `elemento.getCodigo()` en `buscarPorCodigo()` sin importar si el objeto es una `Categoria`, un `Articulo`, o un subtipo de `Articulo`.

3. Herencia para determinar el tipo de artículo

La cadena de herencia es la siguiente:

```
Identificable (interfaz)
  ↑
Articulo      (implements Identificable → tiene getCodigo())
  ↑
┌───────────┴───────────┐
ArticuloAlimenticio      ArticuloElectronico
(IVA 21%)                 (IVA 10.5%)
```

Ambas subclases extienden `Articulo` y agregan comportamiento específico mediante `Calculable`:

```
// ArticuloAlimenticio.java
public class ArticuloAlimenticio extends Articulo implements Calculable {
```

```

public class ArtículoAlimenticio extends Artículo implements Calculable {
    private static final double IVA = 0.21;

    public double calcularPrecioFinal() {
        return getPrecio() * (1 + IVA); // hereda getPrecio() de Artículo
    }
}

// ArtículoElectronico.java
public class ArtículoElectronico extends Artículo implements Calculable {
    private static final double IVA = 0.105;

    public double calcularPrecioFinal() {
        return getPrecio() * (1 + IVA);
    }
}

```

¿Cómo se determina el tipo en tiempo de ejecución? Por polimorfismo:

```

Repositorio<Articulo> repo = new Repositorio<>();
repo.agregar(new ArtículoAlimenticio(1, "Leche", 500, "...", cat));
repo.agregar(new ArtículoElectronico(2, "Mouse", 3000, "...", cat));

Articulo a = repo.buscarPorCodigo(1);
// a es un ArtículoAlimenticio en tiempo de ejecución
// a.toString() imprime "ArticuloAlimenticio { ... (IVA 21%) }"

```

El repositorio trabaja con el tipo base `Articulo`, pero cada objeto recuerda su tipo real. Cuando llamas `toString()` o `calcularPrecioFinal()`, Java ejecuta la versión del subtipo concreto — eso es **dispatch dinámico**.

Resumen del flujo:

1. `Identificable` → contrato mínimo para el repositorio (`getCodigo()`)
2. `Articulo` → implementa ese contrato + datos comunes
3. `ArticuloAlimenticio` / `ArticuloElectronico` → heredan de `Articulo` y especializan comportamiento (IVA diferente)
4. `Repositorio<Articulo>` → puede guardar cualquier subtipo gracias al polimorfismo